DOCUMENTO DE ESTUDIO · PROYECTO FLUTTER

MediTurnos

Análisis de la estructura del proyecto

Estructura de archivos · Arquitectura · API y base de datos · Limpieza

Base del análisis: mediturnos_flutter.zip (41 archivos Dart en lib/, 3.663 líneas).
Fecha: 6 de noviembre de 2026.
Método: lectura estática del código y de los archivos de configuración. El proyecto no fue compilado ni ejecutado; lo que se afirma sobre comportamiento surge de leer el código.

Contenido

1. Panorama general 3

2. Estructura de archivos 3

2.1 Vista general del ZIP . . . 3

2.2 Dos proyectos anidados: envoltorio e interno . . . 3

2.3 Directorios de la raíz y qué representan . . . 4

2.4 El proyecto real: mediturnos/ . . . 4

2.5 Detalle de lib/ archivo por archivo . . . 5

3. Arquitectura aplicada 7

3.1 Patrones y decisiones . . . 7

3.2 Arranque y árbol de Providers . . . 8

3.3 Flujo de ejemplo: solicitar un turno . . . 8

4. API y conexión con la base de datos 9

4.1 Componentes de acceso a datos . . . 9

4.2 Contrato RemoteDataSource . . . 9

4.3 Cómo simula el servidor el MockRemoteDataSource . . . 10

4.4 Base local SQLite (LocalDb) . . . 10

4.5 Autenticación y JWT . . . 10

4.6 Lectura y escritura según haya red . . . 10

4.7 SyncManager: cola y reintentos . . . 11

4.8 Pasos para conectar la API real . . . 11

5. Qué se podría eliminar para limpiar el proyecto 12

5.1 Resumen priorizado . . . 12

5.2 Unificar el proyecto duplicado . . . 12

5.3 Estructura resultante . . . 13

6. Puntos para analizar 13

7. Glosario 13

1. Panorama general

MediTurnos es una aplicación móvil (Android e iOS) para que un paciente solicite, consulte y cancele turnos médicos, con enfoque **offline-first**: si no hay conexión, los turnos se guardan en el dispositivo y se envían al servidor cuando vuelve la red. Solo está implementado el rol *paciente*; si se inicia sesión con otro rol (especialista, secretario, administrador) la app muestra un aviso.

Aspecto	Detalle
Framework / lenguaje	Flutter · Dart (SDK `>=3.3.0 <4.0.0`)
Arquitectura	MVVM + Clean Architecture (Presentation / Domain / Data) con inyección de dependencias manual
Gestión de estado	`provider ^6.1.2` con `ChangeNotifier`
Base de datos local	`sqflite ^2.3.3+1` + `path ^1.9.0` (archivo `mediturnos.db`)
Red / seguridad	`connectivity_plus ^6.0.5` (estado de red) · `flutter_secure_storage ^9.2.2` (JWT)
Solo para el mock	`crypto ^3.0.3` (hash sha256 de contraseñas simuladas)
Interfaz	Material 3 · tipografía Sora vía `google_fonts ^6.2.1`
Backend	Aún no existe: la app corre contra un servidor simulado en memoria (`kUsarMock = true`)

Hallazgos principales

- **Hay dos proyectos Flutter anidados.** El de la raíz es un envoltorio de 5 líneas; todo el código está en la subcarpeta `mediturnos/` (sección 2.2).
- **La arquitectura está bien separada** en tres capas, y el dominio no importa Flutter, SQLite ni HTTP (sección 3).
- **La API REST todavía no está desarrollada.** Existe el contrato (`RemoteDataSource`), un mock que lo cumple y la base local SQLite (sección 4).
- **Casi todo el peso del ZIP es basura regenerable:** ≈109 MB sin comprimir, de los cuales ≈107 MB son la carpeta `.widget_preview/`. El código real son ≈0,1 MB (sección 5).

2. Estructura de archivos

2.1 Vista general del ZIP

```
mediturnos_flutter/                  Raíz del ZIP: proyecto "envoltorio" (casi vacío)
├── lib/main.dart                    5 líneas: llama a app.main() del paquete interno
├── pubspec.yaml                     Depende del paquete real mediante path: ./mediturnos
├── test/widget_test.dart            Copia de las pruebas del paquete interno
├── android/  ios/  web/  windows/   Carpetas de plataforma del envoltorio
├── android_old/                     Android anterior (respaldo): residuo
├── .widget_preview/                 Caché de Widget Preview (≈107 MB, generado)
├── .idea/  *.iml  .metadata         Archivos de Android Studio y de Flutter
├── analysis_options.yaml  .gitignore Reglas de lint y exclusiones de Git
├── README.md                        Plantilla por defecto de Flutter
└── mediturnos/                      PROYECTO REAL: aquí vive toda la aplicación
    ├── lib/                         Código fuente (41 archivos Dart, 3.663 líneas)
    ├── test/widget_test.dart        3 pruebas unitarias
    ├── docs/PROGRESO.md             Estado por requerimiento y pendientes
    ├── android/  ios/  linux/  macos/  Carpetas de plataforma del paquete
    ├── pubspec.yaml                 Dependencias reales de la app
    └── README.md                    Guía de uso, credenciales de prueba y estructura
```

2.2 Dos proyectos anidados: envoltorio e interno

El proyecto de la raíz (`mediturnos_flutter`) no contiene lógica: su `lib/main.dart` solo ejecuta `app.main()` del paquete `mediturnos`, que declara como dependencia local. Es el resultado de haber generado dos veces las carpetas nativas (el README interno indica correr `flutter create .` dentro de `mediturnos/`) y haber abierto la carpeta exterior en el IDE.

	Exterior (mediturnos_flutter)	Interior (mediturnos)
Nombre en pubspec	`mediturnos_flutter`	`mediturnos`
Código en lib/	1 archivo (5 líneas)	41 archivos (3.663 líneas)
Paquete Kotlin (MainActivity)	`com.example.mediturnos_flutter`	`com.mediturnos.mediturnos`
Etiqueta en AndroidManifest	mediturnos_flutter	mediturnos
Plataformas	android, ios, web, windows	android, ios, linux, macos
Pruebas	widget_test.dart	widget_test.dart (idéntico salvo saltos de línea)

> **Por qué importa**
>
> Hay dos `android/`, dos `ios/`, dos `pubspec.yaml` y dos identificadores de aplicación distintos. Por las fechas de modificación, parece que el proyecto exterior es el que se abre y compila en Android Studio (su `.idea` y su caché de Gradle son los más recientes), mientras que el README interior describe la identidad que se quería (`com.mediturnos`). La recomendación de unificar está en la sección 5.2.

2.3 Directorios de la raíz y qué representan

Directorio / archivo	Qué contiene	Tipo
`lib/`, `test/`	main.dart de envoltorio y una copia de las pruebas	Código (duplicado)
`android/`	Proyecto Gradle (Kotlin DSL): build.gradle.kts, AndroidManifest, MainActivity.kt, íconos mipmap, estilos de lanzamiento. Sus subcarpetas `.gradle/` y `.kotlin/` son caché de compilación.	Plataforma
`ios/`	Proyecto Xcode: Runner, Info.plist, AppDelegate, storyboards, íconos. `Flutter/ephemeral` es generado.	Plataforma
`web/`, `windows/`	index.html y manifest PWA; runner C++ con CMake. La app no apunta a estas plataformas.	Plataforma (sin uso)
`android_old/`	Android previo, con tres cachés de Gradle (8.10.2, 8.14, 9.3.1) y dos MainActivity de paquetes distintos.	Residuo
`.widget_preview/`	Mini proyecto Flutter autogenerado por la herramienta Widget Preview: compilados .dill (97 MB en un solo archivo), pubspec.lock, web/.	Generado
`.idea/`, `*.iml`	Configuración de Android Studio: librerías, run configurations, workspace.xml.	IDE
`.metadata`, `.flutter-plugins-dependencies`	Metadatos que escribe la herramienta de Flutter.	Generado
`analysis_options.yaml`	Activa flutter_lints y excluye de análisis las carpetas de plataforma.	Config
`mediturnos/`	El proyecto real (2.4).	Código

2.4 El proyecto real: mediturnos/

Directorio	Qué representa
`lib/`	Toda la aplicación (detalle en 2.5).
`docs/PROGRESO.md`	Estado por requerimiento (Req. 1–15), checklist de la entrega y lista de pendientes: backend real, renovación de JWT, roles no-paciente, pruebas.
`test/widget_test.dart`	Tres pruebas: `Paciente.partirNombre`, formato de `Fechas` y regla `Turno.esProximo`.
`android/`, `ios/`	Carpetas nativas generadas por `flutter create`; solo habría que tocarlas para permisos (ej. INTERNET), íconos o identificador de la app.
`linux/`, `macos/`	Carpetas nativas de escritorio generadas por defecto; sin uso en el alcance móvil.
`pubspec.yaml`	Dependencias reales de la app (sección 1).
`README.md`	Cómo abrir y probar el prototipo, usuarios de prueba y estructura. Su primer párrafo quedó desactualizado: dice que faltan android/ e ios/, que ya existen.

2.5 Detalle de lib/ archivo por archivo

```
lib/
├── main.dart            Punto de entrada: crea Di, ejecuta MediTurnosApp
├── app.dart             MaterialApp, MultiProvider global y _RootGate (splash → login → home)
├── core/                Transversal: DI, configuración, errores, tema, red
├── domain/              Reglas de negocio (Dart puro)
│   ├── entities/        Modelos del negocio
│   ├── repositories/    Interfaces (contratos)
│   └── usecases/        Un caso de uso por acción del usuario
├── data/                Acceso a datos e implementaciones de los contratos
│   ├── datasources/     SQLite, contrato remoto y servidor simulado
│   └── repositories/    Implementaciones y cola de sincronización
└── presentation/        Interfaz y estado de pantalla
    ├── screens/         Vistas (10)
    ├── viewmodels/      ChangeNotifiers (5)
    └── widgets/         Componentes reutilizables (2 archivos)
```

core/ — transversal

Archivo	Responsabilidad
config.dart	`kUsarMock` (interruptor mock / API real) y `kApiBaseUrl` (URL base placeholder de Azure App Service).
connectivity_service.dart	`ConnectivityService`: estado de red real (connectivity_plus) más un modo "sin conexión" simulado. Expone `online` y el stream `onChange`.
di.dart	Clase `Di`: ensamblaje manual (composition root). Crea LocalDb, el origen remoto, SyncManager, los 2 repositorios y los 14 casos de uso.
errors.dart	Jerarquía `AppException`: SinConexion, HorarioNoDisponible, CredencialesInvalidas, UsuarioExistente, Validacion.
fechas.dart	Utilidades de fecha: formato ISO `yyyy-MM-dd`, nombres de meses y días, formatos para mostrar.
safe_notifier.dart	`ChangeNotifier` que ignora `notifyListeners()` después de `dispose()` (evita errores en operaciones asíncronas tardías).
theme.dart	`AppColors` y `buildTheme()` (Material 3, tipografía Sora).

domain/ — reglas de negocio

Archivo	Responsabilidad
entities/paciente.dart	`Paciente`: DNI, nombre, contacto, obra social, `usuarioId`; `copyWith` y JSON.
entities/especialista.dart	`Especialidad` y `Especialista` (con `nombreCompleto` e `iniciales`).
entities/turno.dart	Enum `EstadoTurno` (activo, cancelado, reprogramado, pendienteSync, rechazado) y `Turno`, con conversión a/desde mapa SQLite.
entities/session.dart	Enum `Rol`, `Usuario` y `Session` (JWT + usuario + paciente).
entities/disponibilidad.dart	`Disponibilidad` (horarios libres por día, indicador `desdeCache`) y `SyncResultado`.
repositories/auth_repository.dart	Interfaz de autenticación y perfil.
repositories/turno_repository.dart	Interfaz de catálogo, disponibilidad, solicitar/cancelar, historial, sincronización y borrador del formulario.
usecases/auth_usecases.dart	6 casos de uso (IniciarSesión, RegistrarPaciente, BuscarPaciente, ActualizarPerfil, RestaurarSesión, CerrarSesión) con validaciones de entrada.
usecases/turno_usecases.dart	8 casos de uso. Reglas: no pedir turnos en el pasado, no solapar turnos del mismo paciente, cancelar solo turnos activos o pendientes.

data/ — acceso a datos

Archivo	Responsabilidad
datasources/local_db.dart	`LocalDb`: SQLite (turnos, cola de sincronización, caché clave/valor). Detalle en 4.5.
datasources/remote_data_source.dart	Interfaz `RemoteDataSource`: contrato con la API REST (4.3).
datasources/mock_remote_data_source.dart	Servidor simulado en memoria que respeta el modo offline (4.4).

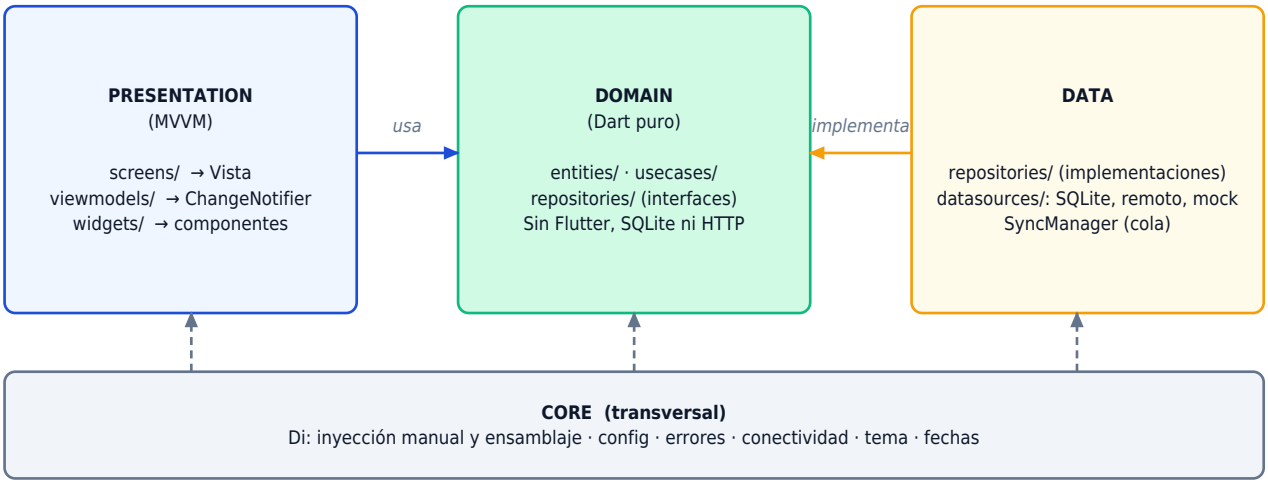
Archivo	Responsabilidad
repositories/auth_repository_impl.dart	Implementa AuthRepository; guarda el JWT en `SecureTokenStorage` y usuario/paciente en la caché local; restaura sesión sin red.
repositories/turno_repository_impl.dart	Único lugar que decide si un dato sale de SQLite o de la API; encola operaciones offline.
repositories/sync_manager.dart	`SyncManager`: reenvía la cola al recuperar conexión, con reintentos y manejo de conflictos (4.8).

presentation/ — interfaz

Archivo	Responsabilidad
screens/login_screen.dart	Ingreso con usuario o DNI y contraseña; "Continuar sin cuenta" (invitado) y acceso al registro.
screens/registro_screen.dart	Alta en dos pasos: usuario/contraseña y datos del paciente, con autocompletado buscando por DNI.
screens/home_shell.dart	Contenedor con barra inferior (Inicio / Mis Turnos / Perfil; el invitado solo ve Inicio). Crea InicioViewModel y TurnosViewModel.
screens/inicio_screen.dart	Listado de profesionales y especialidades con filtros y búsqueda; botón para simular modo sin conexión.
screens/agenda_screen.dart	Calendario mensual con los horarios libres de un especialista.
screens/confirmar_screen.dart	Formulario de confirmación con borrador persistente que sobrevive a pausas del ciclo de vida.
screens/exito_screen.dart	Resultado de la solicitud: "turno confirmado" o "turno guardado" (pendiente de sincronizar).
screens/turnos_screen.dart	Pestañas Próximos / Historial con las tarjetas de turnos.
screens/cancelar_turno_screen.dart	Confirmación de cancelación; delega en TurnosViewModel.
screens/perfil_screen.dart	Ver y editar los datos del paciente (guardar requiere conexión).
viewmodels/*.dart	Auth (sesión, login, registro, perfil, invitado), Inicio (catálogo y filtros), Agenda (mes y selección), Confirmar (envío), Turnos (próximos, historial, cancelar).
widgets/common.dart	403 líneas: 15 componentes reutilizables (Boton, CampoTexto, AppCard, Aviso, BannerOffline, etc.).
widgets/sync_listener.dart	Muestra SnackBars con el resultado de la sincronización y vuelve a sincronizar cuando la app pasa a `resumed`.

3. Arquitectura aplicada

El proyecto combina **MVVM** en la capa de interfaz con **Clean Architecture** en el resto. La idea central es la *regla de dependencia*: el dominio (reglas de negocio) no conoce a nadie; la interfaz y los datos dependen de él.



Regla de dependencia: las flechas continuas apuntan siempre hacia el Dominio.

Figura 1 — Capas y sentido de las dependencias.

Capa	Contiene	Depende de	No debe conocer
Presentation	Vistas, ViewModels, widgets	Domain (casos de uso, entidades) y core	SQLite, HTTP
Domain	Entidades, interfaces de repositorio, casos de uso	Solo core/errors y core/fechas	Flutter, SQLite, HTTP
Data	Repositorios (Impl), orígenes de datos, SyncManager	Domain (implementa sus interfaces) y core	Pantallas, ViewModels
Core	Di, config, tema, errores, conectividad	Todas las capas (Di las ensambla)	—

Comprobado en el código: los archivos de `domain/` no tienen ningún import de `package:flutter`.

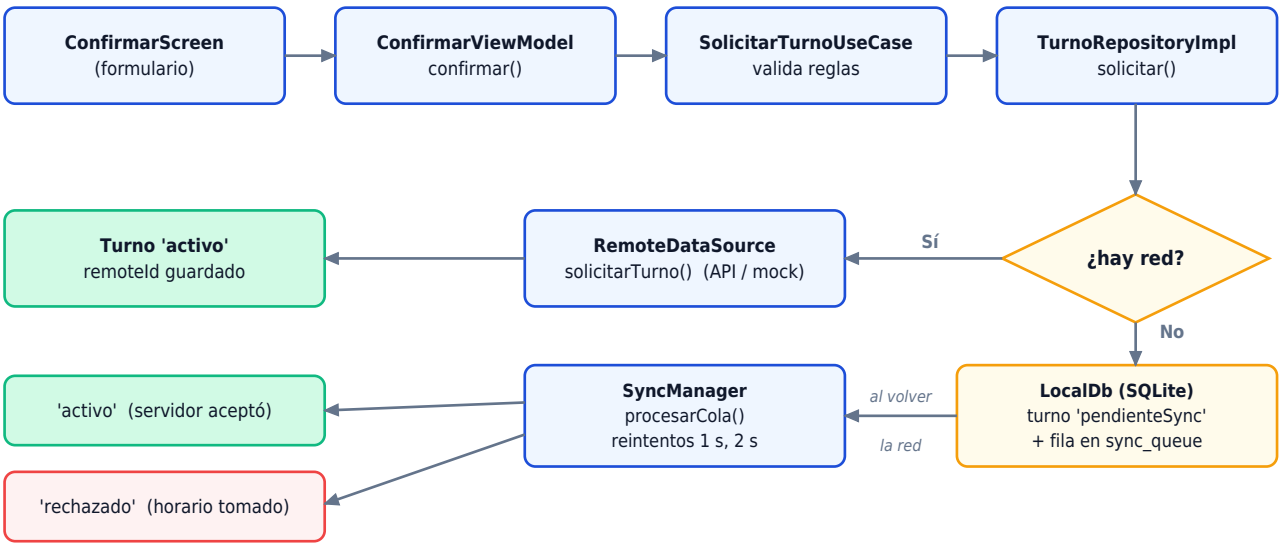
3.1 Patrones y decisiones

Patrón	Dónde	Para qué
MVVM	screens/ (Vista) + viewmodels/ (ChangeNotifier) + Provider	La vista observa campos públicos (`cargando`, `error`, listas) y se redibuja con `notifyListeners()`.
Caso de uso	domain/usecases/	Una clase invocable (`call()`) por acción; concentra validaciones y reglas de negocio.
Repository	domain/repositories/ + data/repositories/	Oculta si el dato viene de SQLite o de la API.
Inversión de dependencias	`RemoteDataSource` y los repositorios abstractos	El resto de la app no sabe si habla con el mock o con la API real.
Inyección manual	`Di` (core/di.dart), expuesta con `Provider<Di>`	Ensamblar objetos sin librerías extra (get_it, etc.).
Offline-first	TurnoRepositoryImpl	Lectura: red primero, caché como respaldo. Escritura: intenta online y, si falla, guarda local y encola.
Cola de salida (outbox)	tabla `sync_queue` + SyncManager	Garantiza que lo hecho sin red se envíe después, en orden.
Caché clave/valor	tabla `cache`	Catálogo, disponibilidad, perfil, sesión y borrador de formulario.
Máquina de estados	`EstadoTurno`	pendienteSync → activo | rechazado; activo → cancelado.
Excepciones de dominio	`AppException` y subclases	Los ViewModels las convierten en mensajes para el usuario.
Observador de ciclo de vida	`WidgetsBindingObserver`	Guarda el borrador del formulario y reintenta la sincronización al volver a primer plano.

3.2 Arranque y árbol de Providers

- `main()` → `WidgetsFlutterBinding.ensureInitialized()` → `Di.crear()` (inicia conectividad, abre LocalDb, elige mock, crea repositorios y casos de uso, y lanza una sincronización si hay pendientes).
- `MediTurnosApp` monta un `MultiProvider` global: `Di`, `ConnectivityService` y `AuthViewModel` (que llama a `restaurar()`).
- `_RootGate` decide la pantalla: spinner mientras restaura, `HomeShell` si hay sesión o invitado, `LoginScreen` si no.
- `HomeShell` crea `InicioViewModel` y `TurnosViewModel` (viven mientras dura la sesión). `AgendaScreen` y `ConfirmarScreen` crean su propio ViewModel local.

3.3 Flujo de ejemplo: solicitar un turno



resultadosSync (stream) → TurnosViewModel refresca la lista y SyncListener muestra un SnackBar.

Figura 2 — Camino de una solicitud, con y sin conexión.

- **Caso de uso:** valida DNI, nombre y correo; que la fecha-hora sea futura; y que el paciente no tenga otro turno activo o pendiente a esa misma hora.
- **Repositorio:** crea un `Turno` local con id `L<microsegundos>` en estado `pendienteSync`. Si hay red, llama al servidor, que revalida la disponibilidad.
- **Sin red (o si se cae a mitad):** comprueba contra la caché local, guarda el turno en SQLite y agrega una fila `'solicitar'` a la cola.
- **Al volver la red:** `SyncManager` envía la cola. Si el servidor acepta, el turno pasa a `activo`; si el horario ya estaba tomado, a `rechazado` y se avisa al paciente.

4. API y conexión con la base de datos

> **Hallazgo clave: la API REST todavía no está desarrollada**
>
> Lo que existe es (a) el **contrato** que la app espera de la API (`RemoteDataSource`), (b) un **servidor simulado en memoria** que lo cumple (`MockRemoteDataSource`) y (c) la **base local SQLite**. No hay cliente HTTP: el paquete `http` no figura en el pubspec y, con `kUsarMock = false`, `Di.crear()` lanza `UnimplementedError`.
>
> La app **nunca se conecta directo a MySQL**. La arquitectura prevista (según config.dart y PROGRESO.md) es: App → API REST (Azure App Service) → MySQL.

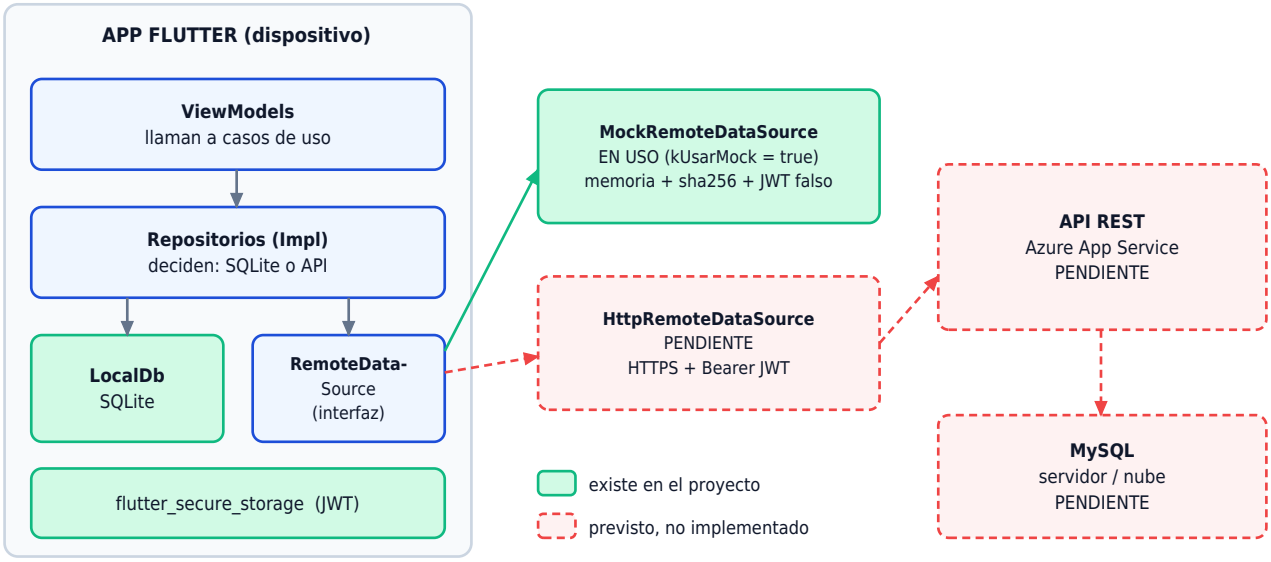


Figura 3 — Estado actual (línea continua) y piezas previstas (línea punteada).

4.1 Componentes de acceso a datos

Componente	Estado	Rol
`LocalDb`	Implementado	SQLite en el dispositivo: turnos propios, cola de sincronización y caché.
`RemoteDataSource`	Implementado (interfaz)	Contrato único con el servidor. Toda implementación debe lanzar `SinConexionException` ante fallas de red.
`MockRemoteDataSource`	Implementado, en uso	Backend simulado para el prototipo y las demostraciones.
`HttpRemoteDataSource`	Pendiente	Cliente real; hoy solo existe un TODO en `di.dart`.
`SecureTokenStorage`	Implementado	Guarda el JWT cifrado (Keychain en iOS, EncryptedSharedPreferences en Android).

4.2 Contrato RemoteDataSource

La cuarta columna es una **propuesta de diseño** para el futuro backend: el código no define endpoints.

Método	Entrada → salida	Comportamiento esperado	Endpoint propuesto
`login`	usuarioODni, password → Session	Valida credenciales; devuelve JWT, usuario y paciente.	POST /auth/login
`buscarPaciente`	dni → Paciente?	Ficha por DNI (autocompleta el registro).	GET /pacientes/{dni}
`registrar`	usuario, password, datos → Session	Crea la cuenta; error si el DNI o el usuario ya existen.	POST /auth/registro
`actualizarPaciente`	Paciente → Paciente	Actualiza la ficha.	PUT /pacientes/{dni}
`especialidades`	→ List<Especialidad>	Catálogo.	GET /especialidades
`especialistas`	→ List<Especialista>	Catálogo.	GET /especialistas
`disponibilidadMes`	espId, año, mes → Map fecha → horas libres	Horarios libres del mes.	GET /especialistas/{id}/disponibilidad?anio=&mes=

Método	Entrada → salida	Comportamiento esperado	Endpoint propuesto
`solicitarTurno`	especialistaId, fecha, hora, paciente → int	Valida en una transacción del servidor; HorarioNoDisponible si ya está tomado. Devuelve el id remoto.	POST /turnos
`cancelarTurno`	remoteId → void	Marca el turno como cancelado.	PATCH /turnos/{id}/cancelar
`turnosPaciente`	dni → List<Turno>	Historial del paciente.	GET /pacientes/{dni}/turnos

4.3 Cómo simula el servidor el MockRemoteDataSource

- **Memoria:** listas en RAM (pacientes, cuentas, turnos). Se pierde todo al cerrar la app.
- **Red simulada:** cada llamada pasa por `_red()`: lanza `SinConexionException` si la app está offline y espera 350 ms.
- **Datos sembrados:** paciente `34123456` con usuario `camilaf_99` / `12345678`; paciente `40111222` sin usuario (para probar el registro); 3 especialidades, 5 especialistas, días de atención por especialista y 6 horarios fijos.
- **Seguridad simulada:** contraseñas con sha256; JWT falso con encabezado HS256, payload `sub/rol/exp` (8 horas) y firma `mock-signature`.
- **Doble reserva:** `solicitarTurno` revalida el horario ("transacción") y lanza `HorarioNoDisponibleException`. Existe `simularReservaAjena()` para provocar el conflicto, pero hoy nadie lo llama.

4.4 Base local SQLite (LocalDb)

Archivo `mediturnos.db`, versión 1, sin migraciones. Tres tablas:

Tabla	Columnas	Uso
`turnos`	id (PK, texto), remote_id, paciente_dni, especialista_id, especialista_nombre, especialidad, fecha, hora, estado	Turnos del paciente. Guarda nombre y especialidad desnormalizados para mostrarlos sin red.
`sync_queue`	id (PK autoincremental), tipo, local_id, payload (JSON), intentos, creado	Operaciones pendientes: `'solicitar'` o `'cancelar'`.
`cache`	key (PK), value (texto JSON)	Claves: `cat:especialidades`, `cat:especialistas`, `disp:{esp}:{año}-{mes}`, `user:usuario`, `user:paciente`, `draft:confirmar`.

Al cerrar sesión, `limpiarDatosUsuario()` borra turnos, cola y las claves `user:*` y `draft:*`, pero conserva el catálogo.

4.5 Autenticación y JWT

- **Login / registro:** el servidor devuelve `Session` (token, usuario, paciente). `AuthRepositoryImpl` guarda el token en el almacenamiento seguro y usuario/paciente en la tabla `cache`.
- **Reabrir sin red:** `restaurarSesion()` lee token y datos locales; no consulta al servidor.
- **Limitaciones actuales:** el token nunca se envía (no hay cliente HTTP), no se verifica su vencimiento y no hay renovación. PROGRESO.md lo lista como pendiente.

4.6 Lectura y escritura según haya red

Operación	Con conexión	Sin conexión
Catálogo	Pide a la API y actualiza la caché.	Lee de la caché (lista vacía si nunca se cargó).
Disponibilidad del mes	Pide a la API y guarda en caché.	Usa la caché y descuenta los turnos propios ya reservados; marca `desdeCache`.
Solicitar turno	API valida en el servidor → `activo`.	Valida contra la caché → `pendienteSync` + cola.
Cancelar turno	API → `cancelado`.	Marca cancelado local + cola. Si el turno nunca llegó al servidor, solo se descarta de la cola.
Historial	Procesa la cola, pide a la API y reemplaza lo sincronizado (conserva pendientes y rechazados).	Lee SQLite.
Perfil, login, registro	API.	Fallan con "Sin conexión". La sesión previa sí se restaura.

4.7 SyncManager: cola y reintentos

- **Cuándo corre:** al volver la conexión (stream `onChange`), al iniciar la app, al abrir el historial y cuando la app pasa a `resumed`.
- **Cómo procesa:** en orden de `id`. Ante falla de red corta el ciclo y deja el resto en cola.
- **Reintentos:** `_conReintentos` hace hasta 3 intentos con espera exponencial (1 s y 2 s entre intentos).
- **Conflicto:** si al sincronizar un `'solicitar'` el servidor responde con un error de aplicación, el turno local pasa a `rechazado` y se informa; la fila sale de la cola.
- **Aviso:** los resultados se emiten por un stream; `TurnosViewModel` refresca y `SyncListener` muestra SnackBars.

4.8 Pasos para conectar la API real

Derivados del código y de PROGRESO.md:

1. Construir el backend (Azure App Service + MySQL) con endpoints equivalentes al contrato. `POST /turnos` debe ser transaccional (unicidad especialista + fecha + hora).
2. Agregar un cliente HTTP al `pubspec.yaml` (por ejemplo `http`); hoy no está declarado.
3. Implementar `HttpRemoteDataSource implements RemoteDataSource`: URL base `kApiBaseUrl`, cabecera `Authorization: Bearer <token>` leyendo el `TokenStorage`, tiempos de espera y conversión JSON ↔ entidades. `Turno` usa `toMap/fromMap` (SQLite), así que necesitará un mapeo JSON propio.
4. Traducir errores: sin red o timeout → `SinConexionException`; 401 → `CredencialesInvalidasException`; 409 → `HorarioNoDisponibleException`; 400/422 → `ValidacionException`.
5. En `Di.crear()` reemplazar el `throw UnimplementedError` por la nueva clase y poner `kUsarMock = false`.
6. Agregar `<uses-permission android:name="android.permission.INTERNET"/>` al AndroidManifest principal (el README ya lo indica; hoy no está).
7. Resolver el vencimiento y la renovación del JWT (pendiente en PROGRESO.md).

5. Qué se podría eliminar para limpiar el proyecto

Todo lo que sigue se puede regenerar (`flutter create .`, `flutter pub get`, el propio IDE) o es redundante. Conviene hacerlo en una rama de Git o con un respaldo, y comprobar después con `flutter analyze` y `flutter test`.

5.1 Resumen priorizado

Elemento	Motivo	Peso	Prioridad
`.widget_preview/`	Autogenerado por Widget Preview; regenerable. Ya figura en .gitignore, pero viaja en el ZIP.	≈107 MB	Alta
`android_old/`	Respaldo de un Android anterior, con tres cachés de Gradle y dos MainActivity de paquetes distintos.	≈0,5 MB	Alta
Proyecto duplicado (envoltorio exterior o interior)	Dos android/, ios/, pubspec e identificadores de app. Dejar solo uno (5.2).	—	Alta
`android/.gradle/`, `android/.kotlin/`	Caché de compilación; android/.gitignore ya la excluye.	≈1,3 MB	Alta (no incluir en entregas)
`.idea/`, `*.iml`, `.flutter-plugins-dependencies`	Configuración local del IDE; se regenera sola.	< 0,1 MB	Media
`local.properties` (×2)	Rutas absolutas del SDK de la PC; no deben compartirse.	mínimo	Media
`ios/Flutter/ephemeral`, `macos/Flutter/ephemeral`, `GeneratedPluginRegistrant.*`	Archivos que Flutter vuelve a escribir en cada compilación.	mínimo	Media
`web/`, `windows/`, `mediturnos/linux/`, `mediturnos/macos/`	Plataformas fuera del alcance (app móvil). `sqflite`, base del modo offline, solo funciona de forma nativa en Android, iOS y macOS.	≈0,4 MB	Media
`test/widget_test.dart` duplicado	Idéntico al interior salvo saltos de línea.	mínimo	Media
`README.md` exterior	Plantilla por defecto de Flutter, sin información del proyecto.	mínimo	Baja
`ios/` (opcional)	Solo si el equipo decide entregar únicamente Android; el README declara Android + iOS.	≈0,1 MB	Según alcance

Código vestigial dentro de lib/

Elemento	Situación	Sugerencia
`MockRemoteDataSource.simularReservaAjena()`	Sin llamadas en todo el proyecto.	Usarlo en una prueba de conflicto de doble reserva o borrarlo.
`LocalDb.cantidadCola()`	Sin llamadas.	Borrarlo, o usarlo para mostrar un contador de pendientes.
`EstadoTurno.reprogramado`	El sistema no modifica turnos: solo los crea y cancela. Aparece en el enum, en una consulta de `local_db.dart` y en el distintivo de `common.dart`.	Quitarlo de los tres lugares, tras confirmar que la API nunca lo devolverá.
`kApiBaseUrl`	Solo aparece en un comentario de `di.dart`.	Conservar: se usará con el cliente HTTP.
`crypto` (dependencia)	Solo la usa el mock.	Se elimina junto con el mock cuando exista la API real.

> **Qué NO eliminar**
>
> `mediturnos/lib/` (toda la aplicación), `docs/PROGRESO.md`, `mock_remote_data_source.dart` (es el backend de la demostración), las pruebas, y el `pubspec.lock`: en una aplicación conviene versionarlo, y el .gitignore interno lo ignora (vale la pena revisarlo).

5.2 Unificar el proyecto duplicado

Lo importante es que quede **un solo** proyecto. Recomendación: **conservar** `mediturnos/` **como raíz**, porque ya tiene el código, el README, las pruebas y el nombre de paquete e identificador que el equipo definió

(`com.mediturnos.mediturnos`); además, el prefijo `com.example` del exterior es un valor de plantilla. El envoltorio no aporta nada.

- Mover `mediturnos/` a un lugar limpio y copiarle el `.gitignore` completo de Flutter del exterior (el interior tiene uno mínimo de 8 líneas).
- Si el repositorio Git cuelga de la raíz exterior, usar `git mv` para conservar el historial.
- Alternativa: conservar el exterior, mover `lib/`, `docs/` y `test/` del interior, fusionar las dependencias en su pubspec y cambiar `package:mediturnos/` por el nombre nuevo en las pruebas (los imports de `lib/` son relativos). Exige además corregir el identificador de la app.

Ejemplo orientativo (Windows / PowerShell)

```
# Ejecutar desde la carpeta que contiene mediturnos_flutter\ (idealmente en una rama o con respaldo)
Copy-Item mediturnos_flutter\.gitignore mediturnos_flutter\mediturnos\.gitignore -Force
Move-Item mediturnos_flutter\mediturnos .\mediturnos
cd mediturnos
Remove-Item -Recurse -Force .idea, linux, macos -ErrorAction SilentlyContinue
flutter clean; flutter pub get; flutter analyze; flutter test
# Cuando todo compile, descartar mediturnos_flutter\ (envoltorio, android_old, .widget_preview, web, windows)
```

5.3 Estructura resultante

```
mediturnos/
├── lib/              core/ · domain/ · data/ · presentation/ · app.dart · main.dart
├── test/             widget_test.dart
├── docs/             PROGRESO.md
├── android/  ios/    Solo las plataformas objetivo
├── pubspec.yaml  pubspec.lock  analysis_options.yaml  .gitignore  README.md (actualizado)
```

Resultado: de ≈109 MB a menos de 1 MB, con una única fuente de verdad.

6. Puntos para analizar

Observaciones técnicas surgidas de la lectura del código, útiles como ejercicio de análisis crítico:

#	Observación	Por qué importa
1	La entidad `Turno` tiene `toMap/fromMap` con columnas SQLite en snake_case.	Un detalle de persistencia entra al dominio; lo más puro sería un mapper en la capa data.
2	`AuthViewModel` e `InicioViewModel` reciben el `Di` completo, mientras Agenda, Confirmar y Turnos reciben solo los casos de uso que necesitan.	Inconsistente: el primer estilo es un service locator y dificulta las pruebas.
3	En la cola, un `'cancelar'` que falla con un error de aplicación incrementa `intentos` pero se reintenta sin límite: ese campo nunca se consulta.	Una operación imposible quedaría reintentándose para siempre.
4	`google_fonts` descarga la tipografía Sora en tiempo de ejecución.	En un primer arranque sin red, la app usa otra fuente; para offline-first conviene empaquetarla como asset.
5	La base se abre con `version: 1` y sin `onUpgrade`.	Cualquier cambio de esquema local exigirá subir la versión y migrar.
6	Solo hay 3 pruebas (entidades y fechas); no hay de casos de uso, repositorios ni SyncManager.	El mock y `simularReservaAjena()` permitirían probar la cola y los conflictos sin servidor.
7	El JWT se guarda pero no se valida ni se renueva.	Al conectar la API real habrá que decidir qué hace la app con una sesión vencida.

7. Glosario

Término	Significado en este proyecto
Offline-first	La app funciona primero contra datos locales y sincroniza con el servidor cuando hay red.
MVVM	Vista (screens) ↔ ViewModel (estado y lógica de pantalla) ↔ Modelo (entidades y casos de uso).
Clean Architecture	Capas concéntricas donde las dependencias apuntan hacia las reglas de negocio.
Caso de uso	Clase que encapsula una acción del usuario y sus reglas (ej. SolicitarTurnoUseCase).
Repository	Abstracción que oculta de dónde vienen los datos (SQLite, API o caché).

Término	Significado en este proyecto
Data source	Acceso directo a una fuente: SQLite local o API remota.
Mock	Implementación falsa de la API, en memoria, para desarrollar sin backend.
Outbox / cola de sincronización	Tabla local con operaciones hechas sin red, enviadas luego y en orden.
Backoff exponencial	Esperar cada vez más entre reintentos (1 s, 2 s, 4 s…).
JWT	Token firmado que emite la API al iniciar sesión e identifica al usuario.
Inyección de dependencias	Entregar a cada clase lo que necesita en vez de que lo cree; aquí lo hace la clase Di.